



XV6-SPIN LOCK

Spinlock structure

```
1 // Mutual exclusion lock.
2 ✓ struct spinlock {
3     uint locked; // Is the lock held?
4
5     // For debugging:
6     char *name; // Name of lock.
7     struct cpu *cpu; // The cpu holding the lock.
8 };
```

- The spinlock structure implements a mutual exclusion lock used to protect shared resources in a multiprocessor environment.
- It prevents multiple CPUs from accessing the same critical section simultaneously, avoiding race conditions.
- **Fields:**
 - **uint locked:**
Indicates whether the lock is currently held.
 - 0 means the lock is free (not held).
 - 1 means the lock is currently acquired by a CPU.
 - **char *name:**
A human-readable name for the lock, useful for debugging and identifying which lock is involved in errors or panics.
 - **struct cpu *cpu:**
A pointer to the CPU structure representing the CPU that currently holds the lock. This helps in debugging and ensuring the lock is not accidentally released by a different CPU.

Xv6 AMOSWAP

- It stands for Atomic Memory Operation Swap
- It will copy a value into a word of memory and at the same time it retrieve the previous value
 - The operation is done without interruption
- Spin lock in Xv6 is defined in "spinlock.c"
- Example: `amoswap.w.aq a5, a5, (s1)`

This instruction atomically swaps the value at address `s1` (which points to `lock`) with the value in register `a5` (which is `1`) and returns the previous value into `a5`.

- The atomic swap ensures no other CPU can modify `lock` between reading and writing it.
- The spin loop continues until the previous value was `0`, meaning the lock was free.
- `__sync_lock_release(&lock)` compiles into:

```
11 void
12 initlock(struct spinlock *lk, char *name)
13 {
14     lk->name = name;
15     lk->locked = 0;
16     lk->cpu = 0;
17 }
```

Parameters

- `struct spinlock *lk`: Pointer to the spinlock to initialize.
- `char *name`: A descriptive name for debugging purposes.

Key Points

- **`lk->locked = 0`**: Sets the lock state to **unlocked**.
- **`lk->cpu = 0`**: No CPU owns the lock yet.
- **`lk->name = name`**: Assigns a human-readable name for debugging/tracing.

When is it used?

- Before any spinlock is used (e.g., in `main()`, during subsystem initialization).
- Ensures the lock is in a known, safe state before use.

acquire

```
19 // Acquire the lock.
20 // Loops (spins) until the lock is acquired.
21 void
22 acquire(struct spinlock *lk)
23 {
24     push_off(); // disable interrupts to avoid deadlock.
25     if(holding(lk))
26         panic("acquire");
27
28     // On RISC-V, sync_lock_test_and_set turns into an atomic swap:
29     // a5 = 1
30     // s1 = &lk->locked
31     // amoswap.w.aq a5, a5, (s1)
32     while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
33         ;
34
35     // Tell the C compiler and the processor to not move loads or stores
36     // past this point, to ensure that the critical section's memory
37     // references happen strictly after the lock is acquired.
38     // On RISC-V, this emits a fence instruction.
39     __sync_synchronize();
40
41     // Record info about lock acquisition for holding() and debugging.
42     lk->cpu = mycpu();
43 }
44
```

- Disable interrupts: `push_off()`;
Prevents deadlock by disabling interrupts during locking.
- Check ownership: `if(holding(lk))`
`panic("acquire");`
Ensures the current CPU does not already hold the lock.
- Atomic lock acquisition:
`while(__sync_lock_test_and_set(&lk->locked, 1) != 0);`
Spins (busy-waits) until the lock is acquired using atomic swap.
- Memory fence: `__sync_synchronize()`;
Prevents compiler/hardware from reordering memory operations before lock.
- Record lock owner: `lk->cpu = mycpu()`;
For debugging and ownership tracking.

Release operation

```
45 // Release the lock.
46 void
47 release(struct spinlock *lk)
48 {
49     if(!holding(lk))
50         panic("release");
51
52     lk->cpu = 0;
53
54     // Tell the C compiler and the CPU to not move loads or stores
55     // past this point, to ensure that all the stores in the critical
56     // section are visible to other CPUs before the lock is released,
57     // and that loads in the critical section occur strictly before
58     // the lock is released.
59     // On RISC-V, this emits a fence instruction.
60     __sync_synchronize();
61
62     // Release the lock, equivalent to lk->locked = 0.
63     // This code doesn't use a C assignment, since the C standard
64     // implies that an assignment might be implemented with
65     // multiple store instructions.
66     // On RISC-V, sync_lock_release turns into an atomic swap:
67     //   s1 = &lk->locked
68     //   amoswap.w zero, zero, (s1)
69     __sync_lock_release(&lk->locked);
70
71     pop_off();
72 }
```

- Check ownership: `if(!holding(lk))`
`panic("release");`
Prevents other CPUs from releasing the lock.
- Clear ownership info: `lk->cpu = 0;`
- Memory barrier: `__sync_synchronize();`
Ensures all critical section writes are visible before unlock.
- Atomic unlock: `__sync_lock_release(&lk->locked);`
Prevents race conditions by atomically releasing the lock.
- Restore interrupt state: `pop_off();`
Re-enables interrupts if they were previously enabled.

holding

```
74 // Check whether this cpu is holding the lock.
75 // Interrupts must be off.
76 int
77 holding(struct spinlock *lk)
78 {
79     int r;
80     r = (lk->locked && lk->cpu == mycpu());
81     return r;
82 }
83
```

- Returns true if the spinlock is currently locked **and** owned by the calling CPU.
- Ensures that the CPU trying to access or release the lock actually holds it.
- Requires interrupts to be disabled to prevent race conditions during the check.
- Helps prevent errors such as releasing a lock that the CPU does not own.

Spinlocks

- Spinlocks **should not be held for long durations**
- Holding a spinlock **wastes CPU cycles** due to busy-waiting
- For long waits or blocking operations, use **sleep()** / **wakeup()** instead
- Spinlocks are used to **protect critical sections and enforce invariants**

```
acquire(&lock);    // Begin critical section (lock acquired)
```

```
.....// Access or modify shared data safely
```

```
release(&lock);    // End critical section (lock released)
```


Questions

- Can spinlock cause deadlock?
- What is the solution?

- Example

Consider a thread & an interrupt handler

What if interrupts are already disabled

- Interrupts might be prematurely disabled
- Solution use a counter

In acquire() :

increment cnt

Disable interrupt

In release ():

decrement the cnt

if cnt==0

enable

push_off

```
88 void
89 push_off(void)
90 {
91     int old = intr_get();
92
93     // disable interrupts to prevent an involuntary context
94     // switch while using mycpu().
95     intr_off();
96
97     if(mycpu()->noff == 0)
98         mycpu()->intena = old;
99     mycpu()->noff += 1;
100 }
101
```

- Disables interrupts to prevent involuntary context switches during critical sections.
- Keeps track of how many times interrupts have been disabled (nested calls).
- Saves the previous interrupt enabled state (intena) on the first disable.
- Ensures that interrupts remain disabled until all corresponding pop_off() calls.

pop_off

```
102 void
103 pop_off(void)
104 {
105     struct cpu *c = mycpu();
106     if(intr_get())
107         panic("pop_off - interruptible");
108     if(c->noff < 1)
109         panic("pop_off");
110     c->noff -= 1;
111     if(c->noff == 0 && c->intena)
112         intr_on();
113 }
```

- Restores interrupts only when all corresponding push_off() calls have been matched.
- Checks to ensure interrupts are currently disabled; panics if called incorrectly.
- Decrements the nested disable counter (noff).
- Re-enables interrupts if this is the last pop_off() call and interrupts were originally enabled.



Thank You